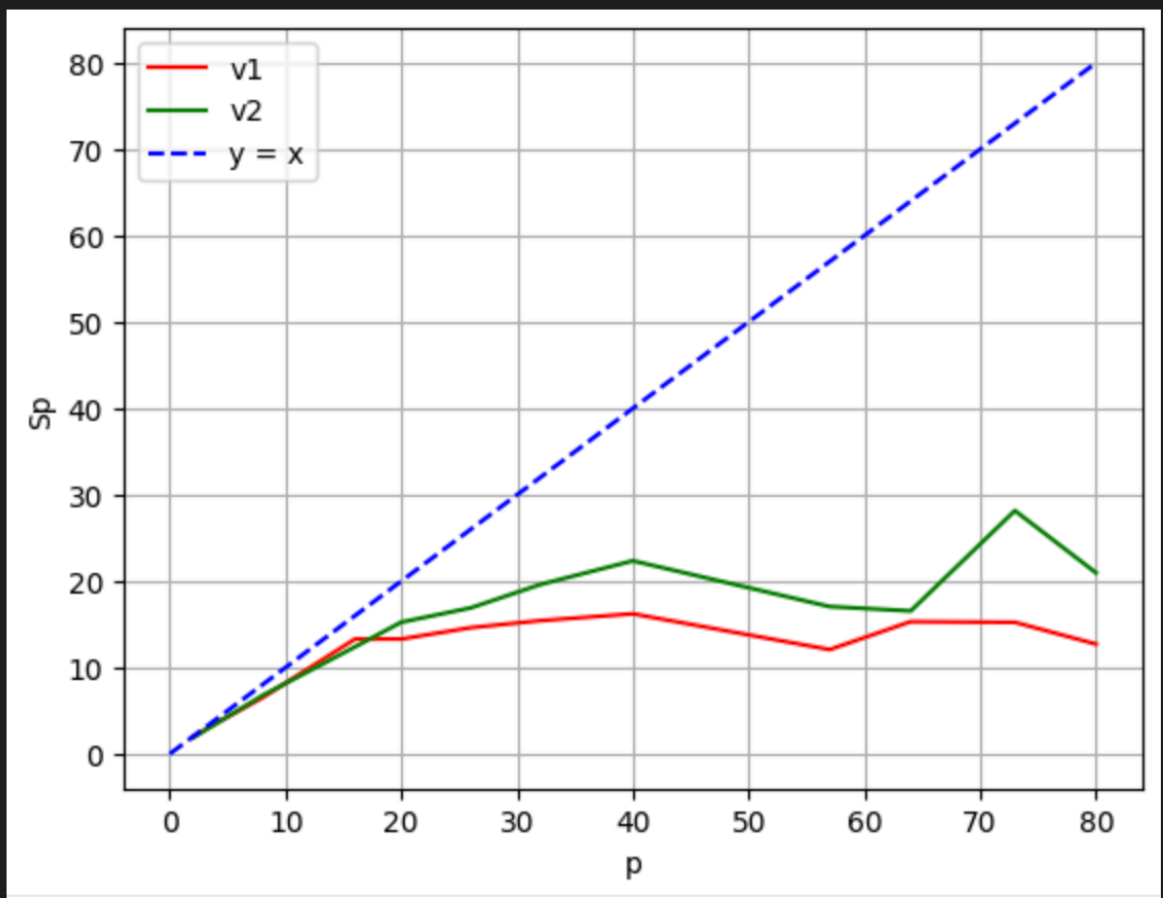
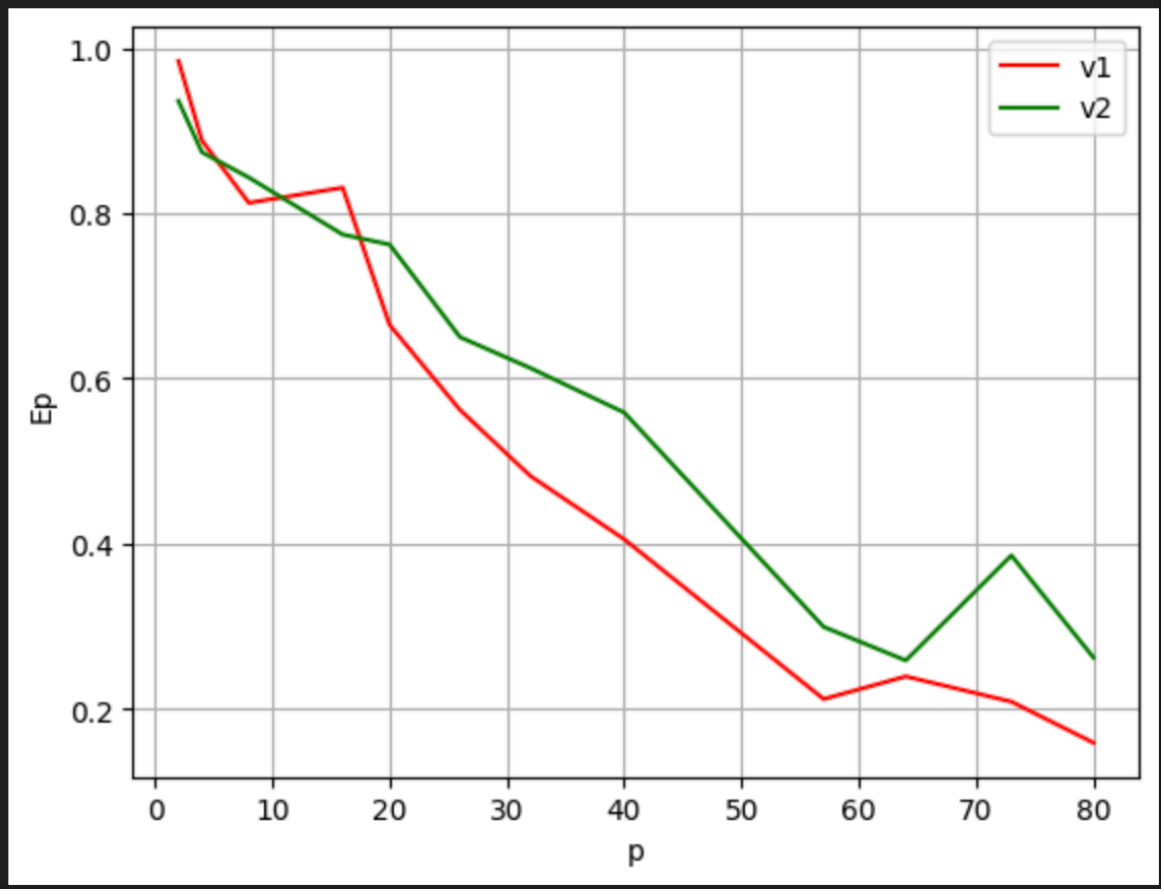
N=5500; eps=1e-5; t=0.01/N	Количество потоков														
	2			4		8		16		20		26		32	
	T1	T2	S2	T4	S4	T8	S8	T16	S16	T20	S20	T26	S26	T32	S32
v1	32.5117	16.5099	1.96923	9.15082	3.55287	5.00049	6.50169	2.44489	13.2978	2.44491	13.2977	2.22524	14.6104	2.10796	15.4233
v2	49.1032	26.2266	1.87227	14.0451	3.4961	7.27777	6.747	3.96314	12.39	3.22102	15.2446	2.90505	16.9027	2.50466	19.6047

40		57		64		73		80	
T40	S40	T57	S57	T64	S64	T73	S73	T80	S80
2.00544	16.2117	2.69313	12.0721	2.12364	15.3094	2.13467	15.2303	2.55766	12.7115
2.19696	22.3505	2.8794	17.0533	2.96735	16.5478	1.74384	28.158	2.34188	20.9674

График ускорения

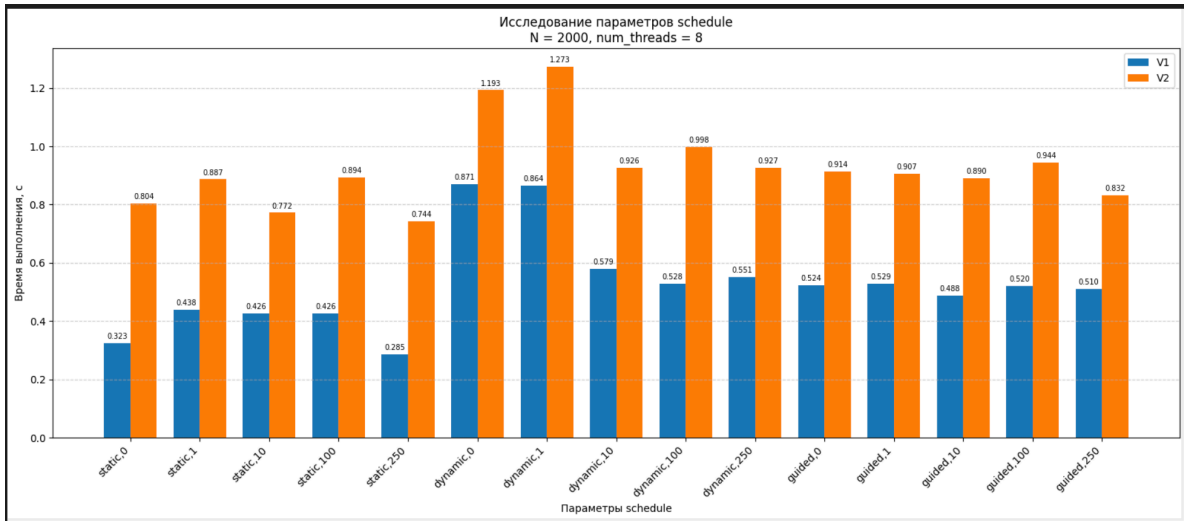


```
График эффективности
```



V1: Максимальное ускорение(16.21) и минимальное время(2) наблюдается при 40 потоках. При дальнейшем увеличении числа потоков время начинает снова расти, это можно объяснить увеличением накладных расходов на синхронизацию потоков и ограничения пропускной способности памяти.
V2: Максимальное ускорение(22.3505) и минимальное время(2.19696) наблюдается так же при 40 потоках. Однако второй вариант работает медленнее первого, это можно объяснить наличием большего числа синхронизаций между потоками. У второго варианта ускорение выше, это говорит о его лучшей масштабируемости.

По графику эффективности видно, что у обоих вариантов с увеличением числа потоков эффективность уменьшается. Связанно это с ростом накладных расходов на синхронизацию, обмен данными и доступ к памяти.



В обоих вариантах лучший результат показал режим (static, 250). Наихудшие результаты показал dynamic, потому что в данной задаче каждая итерация цикла имеет примерно одинаковую вычислительную нагрузку, и динамическое распределение не дает преимуществ.

Вывод:
В данной работе лучше себя показал первый вариант реализации. Он обеспечивает меньшее время выполнения, более стабильные результаты и в исследовании параметров schedule показал лучшие характеристики.